

Information and Network Security

Answers to the Most Frequently Asked Questions – Mumbai University BSc CS Sem 5 | Prepared for Sagar Salunkhe

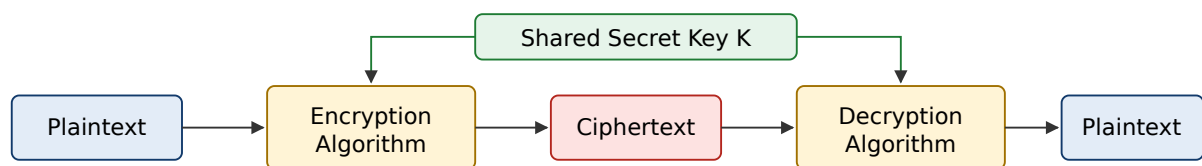
Contents:

1. Symmetric vs Asymmetric cryptography; Active & Passive attacks
2. Substitution vs Transposition; Caesar, Rail Fence, Playfair, Columnar
3. RSA and Diffie-Hellman (numerical)
4. Feistel structure, DES and AES
5. Message Authentication; Hash functions (SHA, HMAC)
6. Digital Signatures; X.509 Certificate
7. PKI and Kerberos
8. PGP and S/MIME
9. Firewalls – types and limitations
10. Malicious software, viruses, life cycle, countermeasures
11. IDS, intrusion techniques, Honeypots
12. IPSec architecture and modes
13. SET protocol and SSL
14. Cryptanalysis, Brute force, Avalanche effect

Q1. Differentiate between Symmetric and Asymmetric cryptography. What are active and passive attacks?

Symmetric Key Cryptography

In symmetric (conventional / secret-key) cryptography, the **same secret key** is used by both sender and receiver for encryption and decryption. Examples: DES, 3DES, AES, Blowfish, RC4.

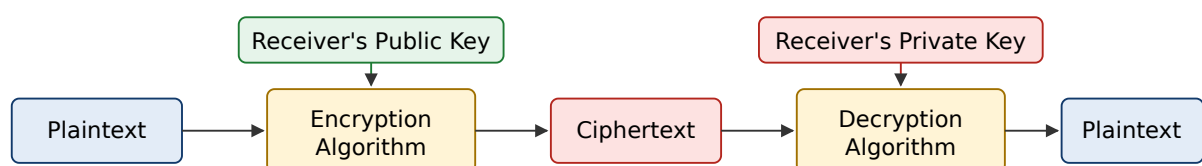


Same key K is used for encryption and decryption

Fig 1.1 – Symmetric encryption model

Asymmetric Key Cryptography

In asymmetric (public-key) cryptography, each user has a **pair of keys** – a public key (known to all) and a private key (kept secret). What one key encrypts, only the other can decrypt. Examples: RSA, Diffie-Hellman, ECC, DSA.



Different keys: encrypt with public key, decrypt with private key

Fig 1.2 – Asymmetric (public-key) encryption model

Difference between Symmetric and Asymmetric Cryptography

Basis	Symmetric	Asymmetric
Keys	One shared secret key	Key pair: public + private
Speed	Very fast; suited for bulk data	Slow (heavy mathematics)
Key size	Small (56, 128, 256 bits)	Large (1024–4096 bits for RSA)
Key distribution	Difficult – key must be shared secretly	Easy – public key can be published
Number of keys for N users	$N(N-1)/2$	$2N$
Services	Confidentiality only	Confidentiality, authentication, non-repudiation, digital signatures, key exchange
Based on	Substitution & permutation	Mathematical hard problems (factoring, discrete log)
Examples	DES, 3DES, AES, RC4, Blowfish	RSA, Diffie-Hellman, ECC, DSA, ElGamal

Security Attacks

A security attack is any action that compromises the security of information. They are classified as **passive** and **active** attacks.

1. Passive Attacks – The attacker only *monitors / eavesdrops* on transmissions; data is not altered. Goal: obtain information. They are **difficult to detect**, so the emphasis is on **prevention** (encryption).

- **Release of message contents:** reading the contents of an email, file or phone call.
- **Traffic analysis:** even if data is encrypted, the attacker studies the pattern – who is communicating, how often, message lengths – to guess the nature of communication.

2. Active Attacks – The attacker *modifies* the data stream or creates false streams. They are **easy to detect but hard to prevent**; the emphasis is on detection and recovery.

- **Masquerade:** one entity pretends to be another (e.g., using stolen passwords).
- **Replay:** capturing a valid message and retransmitting it later to produce an unauthorized effect.
- **Modification of messages:** altering, delaying or reordering a legitimate message ("Pay A ₹100" → "Pay B ₹1000").
- **Denial of Service (DoS):** preventing normal use of communication facilities, e.g., flooding a server.

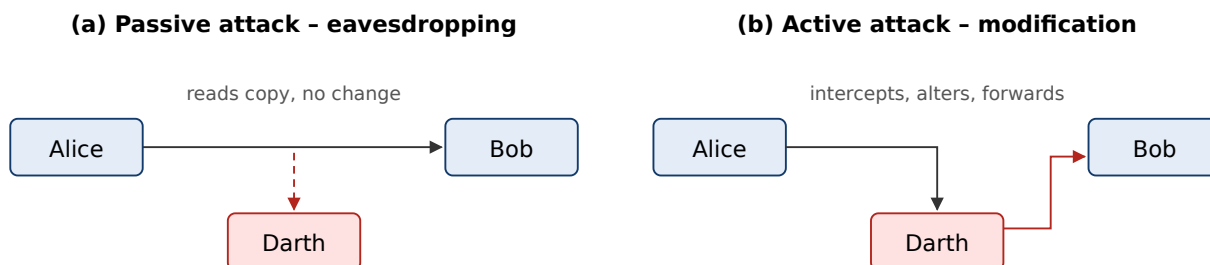


Fig 1.3 – Passive vs active attacks

Passive Attack	Active Attack
Only observes / copies data	Modifies data or disrupts system
System resources not affected	System resources affected
Hard to detect	Easier to detect
Prevention via encryption	Detection and recovery
Threat to confidentiality	Threat to integrity & availability
Eg: eavesdropping, traffic analysis	Eg: masquerade, replay, modification, DoS

Q2. Substitution vs Transposition techniques; Caesar, Rail Fence, Playfair, Columnar transposition

Substitution Technique

Each letter (or group of letters) of the plaintext is **replaced** by another letter, number or symbol. The *identity* of characters changes but their *position* stays the same. Examples: Caesar, Monoalphabetic, Playfair, Hill, Vigenère, One-time pad.

Transposition Technique

The letters of plaintext are **rearranged** (permuted) without changing them. The *position* of characters changes, their *identity* remains the same. Examples: Rail Fence, Columnar (row) transposition.

Substitution	Transposition
Characters are replaced by other characters	Characters are rearranged (permuted)
Position stays same; identity changes	Identity stays same; position changes
Letter frequencies change (shifted)	Letter frequencies remain same as plaintext
Provides <i>confusion</i>	Provides <i>diffusion</i>
Caesar, Playfair, Hill, Vigenère	Rail Fence, Columnar transposition

(a) Caesar Cipher

Each letter is shifted by a fixed number k (key) positions. Letters A-Z $\rightarrow$ 0-25.

Encryption: $C = E(k, p) = (p + k) \bmod 26$

Decryption: $p = D(k, C) = (C - k) \bmod 26$

Example: Plaintext = HELLO WORLD, key $k = 3$

Plain : H E L L O W O R L D

+3 : K H O O R Z R U O G

Ciphertext = KH00R ZRU0G

Decryption: shift each letter back by 3 $\rightarrow$ HELLO WORLD

Weakness: only 25 possible keys, so it is easily broken by brute force.

(b) Rail Fence Cipher

Plaintext is written diagonally (zig-zag) over a number of rows (depth) and read row-by-row.

Plaintext = MEET ME AFTER THE PARTY, depth = 2

Row 1: M . E . M . A . T . R . H . P . R . Y

Row 2: . E . T . E . F . E . T . E . A . T .

Read row 1 then row 2:

Ciphertext = MEMATRHPRY ETEFETEAT → MEMATRHPRYETEFETEAT

Decryption: length 19 → row 1 gets 10 letters (MEMATRHPRY), row 2 gets 9 (ETEFETEAT); write them back alternately: M E E T M E A F T E R T H E P A R T Y → MEETMEAFETERTHEPARTY

(c) Playfair Cipher

Uses a 5×5 matrix built from a keyword (I and J share a cell). Plaintext is encrypted two letters (digram) at a time.

Rules:

1. Split plaintext into pairs; if both letters in a pair are the same, insert filler 'X'. Add 'X' at the end if odd length.
2. **Same row:** replace each letter by the letter to its **right** (wrap around).
3. **Same column:** replace each letter by the letter **below** it (wrap around).
4. **Otherwise (rectangle):** replace each letter by the letter in its own row but in the other letter's column.

Example: Key = **MONARCHY**, Plaintext = **INSTRUMENTS**

M	O	N	A	R
C	H	Y	B	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z

Pairs: IN ST RU ME NT SX

IN → rectangle → GA

ST → same row (row 4) → TL

RU → rectangle → MZ

ME → same column (col1) → CL

NT → rectangle → RQ

SX → same column (col4) → XA

Ciphertext = GATLMZCLRQXA

Decryption uses the reverse rules (left in row, up in column, rectangle same).

(d) Columnar (Row) Transposition

Plaintext is written row-by-row in a rectangle whose width equals the key length; columns are read out in the order given by the key.

Key : 4 3 1 2 5 6 7

Plaintext : ATTACK POSTPONED UNTIL TWO AM (padded with XYZ)

Key	→	4	3	1	2	5	6	7
		A	T	T	A	C	K	P
		O	S	T	P	O	N	E
		D	U	N	T	I	L	T
		W	O	A	M	X	Y	Z

Read column 1 (TTNA), 2 (APTM), 3 (TSUO), 4 (AODW), 5 (COIX), 6 (KNLY), 7 (PETZ)

Ciphertext = TTNAAPTMTSUOAODWCOIXKNLYPETZ

Decryption: write ciphertext column-wise in key order (4 letters per column), read row-wise.

Q3. RSA Algorithm and Diffie-Hellman Key Exchange (with numerical examples)

RSA Algorithm

RSA (Rivest, Shamir, Adleman – 1977) is a public-key algorithm whose security depends on the difficulty of **factoring a large number** n into its two prime factors.

Steps:

1. Key Generation

- Select two large prime numbers p and q ($p \neq q$).
- Calculate $n = p \times q$.
- Calculate Euler's totient $\phi(n) = (p - 1)(q - 1)$.
- Select e such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$.
- Calculate d such that $d \times e \equiv 1 \pmod{\phi(n)}$ ($d = e^{-1} \pmod{\phi(n)}$).
- Public key $PU = \{e, n\}$; Private key $PR = \{d, n\}$.

2. **Encryption:** $C = M^e \pmod n$ ($M < n$)

3. **Decryption:** $M = C^d \pmod n$

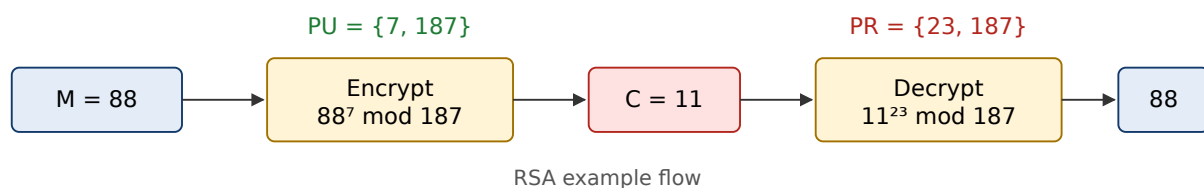


Fig 3.1 – RSA encryption/decryption for the example below

Numerical Example: $p = 17$, $q = 11$, $M = 88$

- $n = p \times q = 17 \times 11 = 187$
- $\phi(n) = (17 - 1)(11 - 1) = 16 \times 10 = 160$
- Choose $e = 7$ ($\gcd(7, 160) = 1$)
- Find d : $d \times 7 \equiv 1 \pmod{160} \rightarrow d = 23$ (check: $23 \times 7 = 161 = 1 \times 160 + 1$ ✓)
- Public key $PU = \{7, 187\}$; Private key $PR = \{23, 187\}$

Encryption: $C = 88^7 \pmod{187}$

$$\begin{aligned} 88^1 \pmod{187} &= 88 \\ 88^2 \pmod{187} &= 7744 \pmod{187} = 77 \\ 88^4 \pmod{187} &= 77^2 \pmod{187} = 5929 \pmod{187} = 132 \\ 88^7 &= 88^4 \times 88^2 \times 88^1 = (132 \times 77 \times 88) \pmod{187} = 894432 \pmod{187} = 11 \\ \rightarrow C &= 11 \end{aligned}$$

Decryption: $M = 11^{23} \pmod{187}$

$$\begin{aligned} 11^1 &= 11, \quad 11^2 = 121, \quad 11^4 = 121^2 \pmod{187} = 55, \quad 11^8 = 55^2 \pmod{187} = 33, \\ 11^{16} &= 33^2 \pmod{187} = 154 \\ 11^{23} &= 11^{16} \times 11^4 \times 11^2 \times 11^1 = (154 \times 55 \times 121 \times 11) \pmod{187} = 88 \\ \rightarrow M &= 88 \quad \checkmark \end{aligned}$$

Diffie-Hellman Key Exchange

Diffie-Hellman (1976) allows two users to **securely agree on a shared secret key** over an insecure channel. It is used only for key exchange, not for encryption. Its security depends on the

difficulty of computing **discrete logarithms**.

Steps:

1. **Global public elements:** a prime number q and α , a primitive root of q .
2. **User A:** selects private $X_A < q$; computes public $Y_A = \alpha^{X_A} \bmod q$.
3. **User B:** selects private $X_B < q$; computes public $Y_B = \alpha^{X_B} \bmod q$.
4. A and B exchange Y_A and Y_B .
5. **A computes** $K = (Y_B)^{X_A} \bmod q$; **B computes** $K = (Y_A)^{X_B} \bmod q$. Both get the same $K = \alpha^{X_A X_B} \bmod q$.

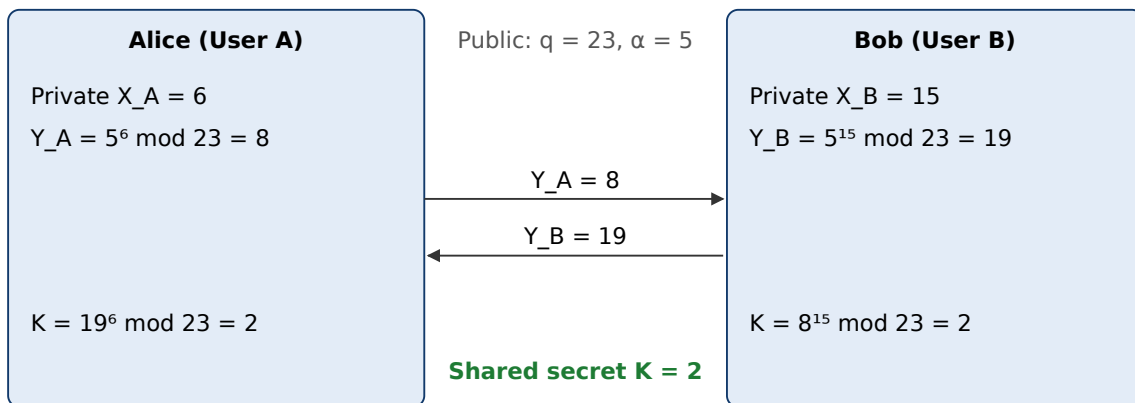


Fig 3.2 - Diffie-Hellman key exchange

```
Numerical Example:  q = 23, α = 5, X_A = 6, X_B = 15
Y_A = 5^6 mod 23 = 15625 mod 23 = 8
Y_B = 5^15 mod 23 = 19
Alice: K = Y_B^X_A mod q = 19^6 mod 23 = 2
Bob  : K = Y_A^X_B mod q = 8^15 mod 23 = 2
Shared secret key K = 2 ✓
```

Weakness: Diffie-Hellman is vulnerable to the **Man-in-the-Middle attack** because the exchanged public values are not authenticated. Solution: use digital signatures / certificates.

Q4. Feistel structure; Notes on DES and AES

Feistel Cipher Structure

The Feistel structure (Horst Feistel, IBM) is a design model for symmetric **block ciphers**. It applies a series of rounds of substitution and permutation, implementing Shannon's ideas of **confusion** and **diffusion**. DES is based on it.

Working:

1. The plaintext block (2w bits) is divided into two halves: **L₀** and **R₀**.
2. For each round $i = 1 \dots n$, using subkey K_i derived from the main key:

$$L_i = R_{i-1} \quad R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

3. After the last round, the two halves are swapped and combined to give the ciphertext.
4. **Decryption** uses the same algorithm with subkeys in reverse order ($K_n \dots K_1$). F need not be invertible.

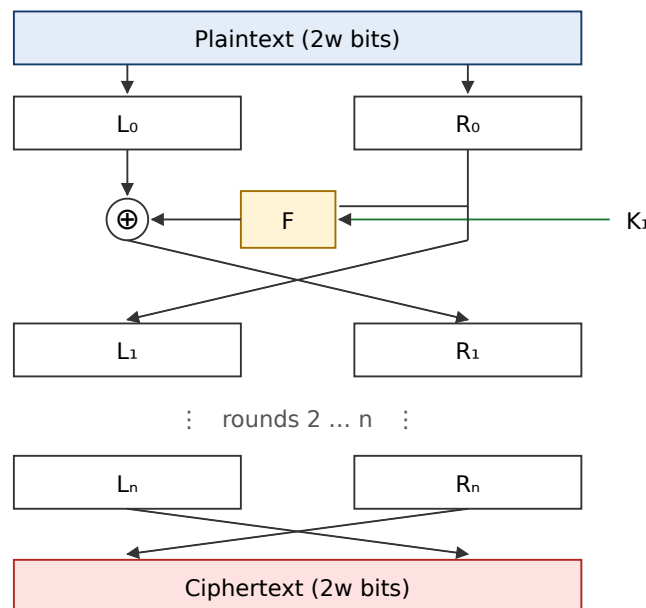


Fig 4.1 – Feistel cipher structure (one round shown)

Design parameters: block size, key size, number of rounds, subkey generation algorithm, round function F, fast software encryption, ease of analysis.

Data Encryption Standard (DES)

- Adopted by NIST (NBS) in 1977; based on IBM's Lucifer cipher; Feistel structure.
- **Block size:** 64 bits; **Key size:** 64 bits of which **56 bits** are effective (8 parity bits); **16 rounds**; 16 subkeys of 48 bits.

Steps of DES:

1. **Initial Permutation (IP)** rearranges the 64-bit plaintext.
2. Block is split into L_0 and R_0 (32 bits each).
3. **16 rounds**, each with the round function F:
 - **Expansion (E):** 32-bit R expanded to 48 bits.

- **XOR** with 48-bit round key K_i .
- **S-boxes**: 8 S-boxes, each maps 6 bits $\rightarrow$ 4 bits (48 $\rightarrow$ 32 bits). S-boxes are the only non-linear part.
- **Permutation (P)**: 32-bit straight permutation.

4. **32-bit swap** of the halves.

5. **Inverse Initial Permutation (IP^{-1})** produces the 64-bit ciphertext.

Key generation: 64-bit key $\rightarrow$ Permuted Choice 1 (56 bits) $\rightarrow$ split into two 28-bit halves $\rightarrow$ left circular shift (1 or 2 bits per round) $\rightarrow$ Permuted Choice 2 $\rightarrow$ 48-bit subkey.

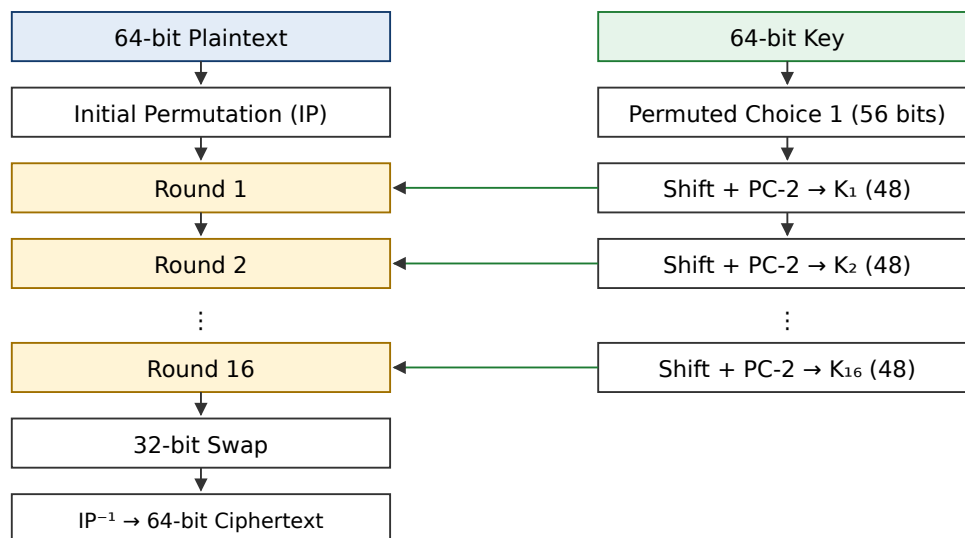


Fig 4.2 – General structure of DES

Weakness: the 56-bit key is too short and can be broken by brute force (EFF "Deep Crack", 1998). Hence 3DES and later AES were adopted.

Advanced Encryption Standard (AES)

- Selected by NIST in 2001; originally the **Rijndael** cipher (Daemen & Rijmen).
- **Not a Feistel cipher** – processes the entire block in parallel using a substitution-permutation network.
- **Block size**: 128 bits (arranged as a 4×4 byte matrix called the *State*).
- **Key sizes / rounds**: 128 bits $\rightarrow$ 10 rounds; 192 bits $\rightarrow$ 12 rounds; 256 bits $\rightarrow$ 14 rounds.

Four transformations in each round:

1. **SubBytes**: byte-by-byte substitution using an S-box (non-linear, gives confusion).
2. **ShiftRows**: row r of the State is circularly shifted left by r bytes (row 0 unchanged).
3. **MixColumns**: each column is multiplied by a fixed matrix in $GF(2^8)$ (diffusion).
4. **AddRoundKey**: State is XORed with the round key.

Structure: initial AddRoundKey $\rightarrow$ (N–1) full rounds $\rightarrow$ final round *without MixColumns*.

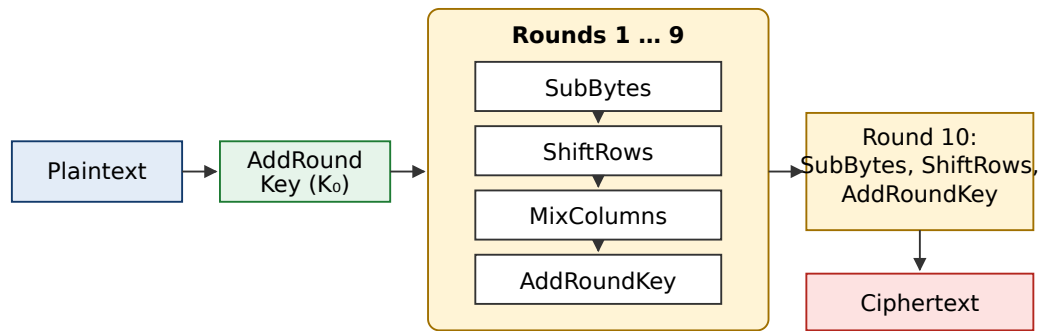


Fig 4.3 - AES-128 encryption structure

DES	AES
64-bit block	128-bit block
56-bit key	128 / 192 / 256-bit key
16 rounds	10 / 12 / 14 rounds
Feistel structure	Substitution-permutation network
Insecure today (brute force)	Secure; current standard

Q5. Message Authentication; characteristics and properties of Hash functions (SHA, HMAC)

Message Authentication

Message authentication is a procedure to verify that a received message **comes from the claimed source** (source authentication) and **has not been altered** (integrity). It may also verify sequencing and timeliness. It protects against masquerade, content modification, sequence modification and timing modification.

Three types of authentication functions:

1. **Message Encryption** - the ciphertext itself serves as the authenticator (only the key holder could have produced it).
2. **Message Authentication Code (MAC)** - a fixed-length value computed as $MAC = C(K, M)$ using a shared secret key, appended to the message. The receiver recomputes and compares it.
3. **Hash Function** - maps a message of any length to a fixed-length hash value $h = H(M)$, which is used as the authenticator (usually protected by encryption or a key).

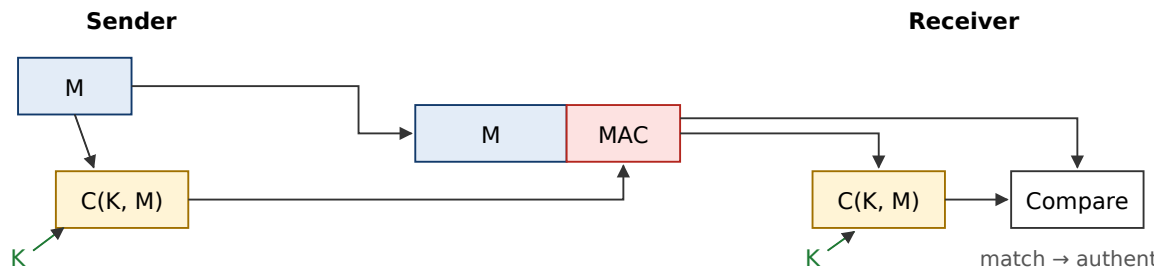


Fig 5.1 - Message authentication using MAC

Hash Function

A hash function H accepts a variable-length message M and produces a fixed-size **message digest** $h = H(M)$. A change of even one bit in M changes the hash completely. Used for integrity checking, digital signatures, password storage.

Requirements / Properties of a cryptographic hash function:

1. **Variable input size:** H can be applied to a block of data of any size.
2. **Fixed output size:** H produces a fixed-length output.
3. **Efficiency:** $H(x)$ is easy to compute for any x .
4. **Pre-image resistant (one-way):** given h , it is infeasible to find x such that $H(x) = h$.
5. **Second pre-image resistant (weak collision resistance):** given x , it is infeasible to find $y \neq x$ with $H(y) = H(x)$.
6. **Collision resistant (strong collision resistance):** infeasible to find any pair (x, y) with $H(x) = H(y)$.
7. **Pseudo-randomness:** output of H looks random (avalanche effect).

Secure Hash Algorithm (SHA)

- Developed by NIST/NSA. Versions: SHA-1 (160-bit), SHA-2 family (SHA-224, SHA-256, SHA-384, SHA-512) and SHA-3.
- **SHA-512 working:**

1. **Append padding bits** so that $\text{length} \equiv 896 \pmod{1024}$.
2. **Append length** - a 128-bit block containing the original message length.
3. **Initialize hash buffer** - eight 64-bit registers (a - h) with fixed initial values.
4. **Process message in 1024-bit blocks** - each block goes through 80 rounds of the compression function F , using word values W_t and constants K_t .
5. **Output** - after the last block, the buffer holds the 512-bit message digest.

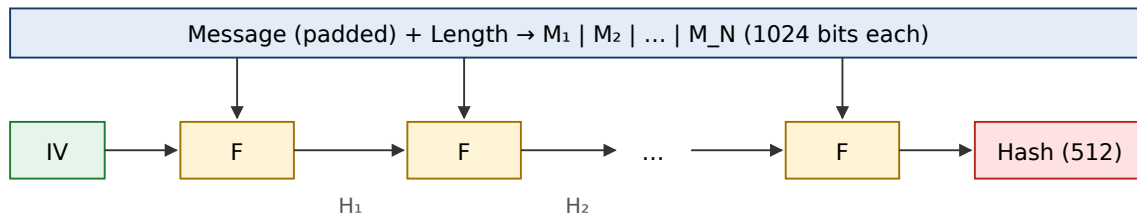


Fig 5.2 - Iterated hash structure of SHA

HMAC (Hash-based MAC)

HMAC builds a MAC from a cryptographic hash function (e.g., SHA-256) and a secret key. It is used in IPSec, SSL/TLS.

$$\text{HMAC}(K, M) = H[(K^+ \oplus \text{opad}) \parallel H[(K^+ \oplus \text{ipad}) \parallel M]]$$

K^+ = key padded with zeros to block size b
 ipad = 00110110 (0x36) repeated $b/8$ times
 opad = 01011100 (0x5C) repeated $b/8$ times

Steps: (1) Pad K to b bits $\rightarrow K^+$. (2) XOR K^+ with $\text{ipad} \rightarrow S_i$. (3) Append M to S_i . (4) Hash it $\rightarrow H(S_i \parallel M)$. (5) XOR K^+ with $\text{opad} \rightarrow S_o$. (6) Append the inner hash to S_o . (7) Hash again $\rightarrow \text{HMAC output}$.

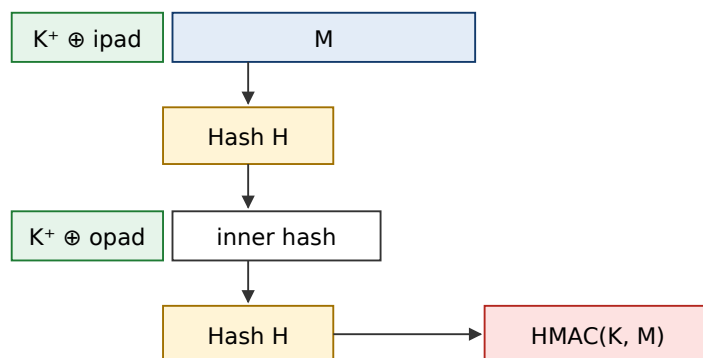


Fig 5.3 - HMAC structure

Advantages of HMAC: uses available hash functions without modification; hash function can be replaced easily; keeps hash performance; security is provably related to the underlying hash.

Q6. Generation and verification of Digital Signatures; components of X.509 Certificate

Digital Signature

A digital signature is an electronic, cryptographic equivalent of a handwritten signature. It provides **authentication**, **integrity** and **non-repudiation**. It is created with the sender's **private key** and verified with the sender's **public key**.

Generation (at sender - Alice):

1. Compute the hash of the message: $h = H(M)$.
2. Encrypt the hash with Alice's private key: $S = E(PR_a, h)$. This is the digital signature.
3. Send M along with S.

Verification (at receiver - Bob):

1. Compute hash of received message: $h' = H(M)$.
2. Decrypt signature using Alice's public key: $h = D(PU_a, S)$.
3. If $h = h' \rightarrow$ signature valid (message authentic and unaltered); otherwise reject.

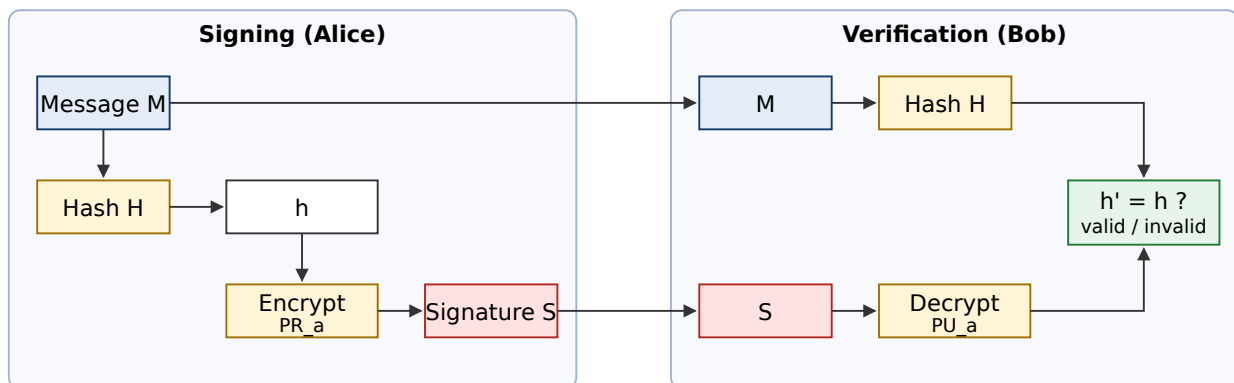


Fig 6.1 - Digital signature generation and verification

Properties: must depend on the message; must use information unique to the sender; easy to produce, recognize and verify; computationally infeasible to forge. Standard: **DSS (Digital Signature Standard)** using DSA, RSA or ECDSA.

X.509 Certificate

X.509 is an ITU-T standard that defines the format of **public-key certificates**. A certificate binds a user's identity to their public key and is signed by a trusted **Certification Authority (CA)**. Used in SSL/TLS, S/MIME, IPsec.

Version (v1, v2, v3)
Certificate Serial Number
Signature Algorithm Identifier
Issuer Name (CA)
Period of Validity (not before / after)
Subject Name
Subject's Public-Key Info
Issuer Unique Identifier (v2)
Subject Unique Identifier (v2)
Extensions (v3)
Signature of CA (on all fields)

Fig 6.2 – X.509 certificate format

Key components:

1. **Version:** 1, 2 or 3 – indicates format.
2. **Serial number:** unique integer assigned by the CA.
3. **Signature algorithm identifier:** algorithm used by CA to sign (e.g., SHA256withRSA).
4. **Issuer name:** X.500 name of the CA that issued and signed the certificate.
5. **Period of validity:** first and last dates of validity.
6. **Subject name:** name of the user/entity to whom the certificate is issued.
7. **Subject's public-key information:** public key of the subject and algorithm identifier.
8. **Issuer / Subject unique identifier:** optional (v2) – to identify issuer/subject uniquely if names are reused.
9. **Extensions (v3):** key usage, certificate policies, alternate names, basic constraints.
10. **Signature:** hash of all other fields encrypted with the CA's private key.

Notation: **CA<<A>>** = certificate of user A issued by CA. Certificates are revoked using a **Certificate Revocation List (CRL)**.

Q7. Public Key Infrastructure (PKI) and Kerberos

Public Key Infrastructure (PKI)

PKI is the set of hardware, software, people, policies and procedures needed to **create, manage, store, distribute and revoke digital certificates** based on asymmetric cryptography. Its main objective is to enable secure, convenient and efficient acquisition of public keys. The IETF PKIX model is based on X.509.

Components of PKIX model:

1. **End Entity:** user, device or process that uses a certificate.
2. **Certification Authority (CA):** issues certificates and CRLs; trusted third party.
3. **Registration Authority (RA):** optional; verifies the identity of end entities on behalf of the CA.
4. **CRL Issuer:** publishes Certificate Revocation Lists.
5. **Repository:** stores certificates and CRLs for retrieval by end entities (e.g., LDAP directory).

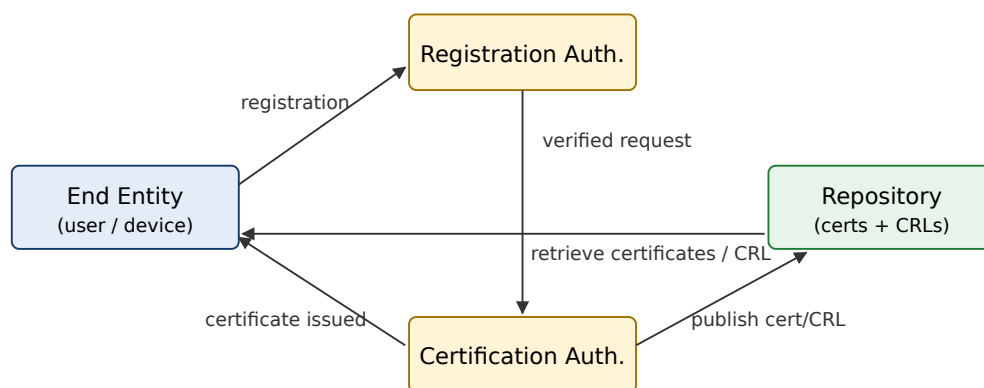


Fig 7.1 – PKIX architecture

PKI management functions: registration, initialization, certification, key-pair recovery, key-pair update, revocation request, cross-certification.

Kerberos

Kerberos (MIT, Project Athena) is a **centralized authentication service** for client/server networks based on **symmetric encryption** and a **trusted third party**. Users prove their identity once and get tickets; passwords are never sent over the network.

Components:

- **Client (C):** the user's workstation.
- **Authentication Server (AS):** knows all users' passwords (keys); verifies user and issues a Ticket-Granting Ticket (TGT).
- **Ticket Granting Server (TGS):** issues service-granting tickets for specific servers.
- **Key Distribution Center (KDC) = AS + TGS.**
- **Application Server (V):** provides the requested service.
- **Tickets & Authenticators:** tickets prove identity; authenticators (with timestamp) prevent replay.

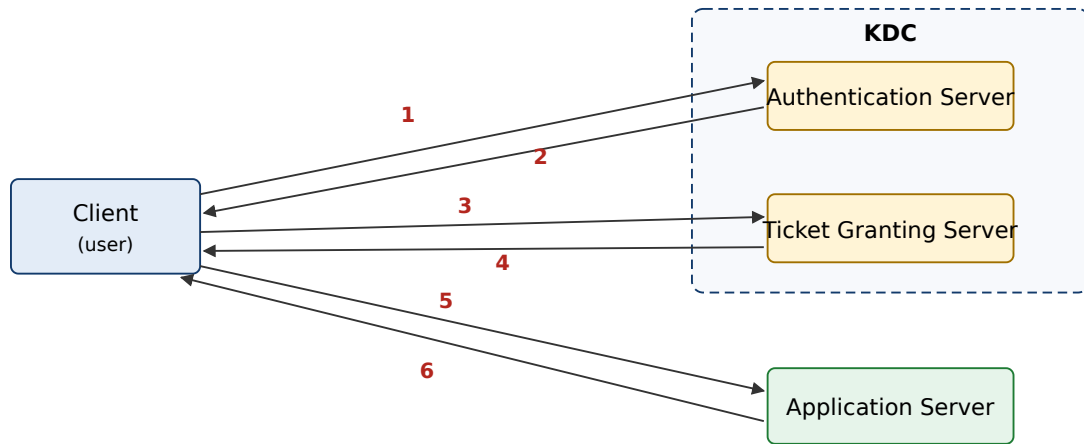


Fig 7.2 - Kerberos authentication exchanges

Authentication process (Kerberos v4 message exchange):

1. **C → AS:** $ID_C \parallel ID_{tgs} \parallel TS_1$ - user logs on and requests a TGT.
2. **AS → C:** $E(K_C, [K_{C,tgs} \parallel ID_{tgs} \parallel TS_2 \parallel Lifetime_2 \parallel Ticket_{tgs}])$ - encrypted with key derived from user's password. Client decrypts it using the password.
3. **C → TGS:** $ID_V \parallel Ticket_{tgs} \parallel Authenticator_C$ - requests a service-granting ticket.
4. **TGS → C:** $E(K_{C,tgs}, [K_{C,v} \parallel ID_V \parallel TS_4 \parallel Ticket_V])$ - grants the service ticket.
5. **C → V:** $Ticket_V \parallel Authenticator_C$ - requests service.
6. **V → C:** $E(K_{C,v}, [TS_5 + 1])$ - optional mutual authentication of server.

Requirements met: Secure, Reliable, Transparent (single sign-on), Scalable.

Q8. How do PGP and S/MIME provide email security?

PGP (Pretty Good Privacy)

PGP, developed by **Phil Zimmermann** (1991), is an open, freely available package that provides **confidentiality and authentication** for email and file storage. It uses the best available algorithms (RSA, DSS, ElGamal, CAST-128, IDEA, 3DES, AES, SHA-1).

PGP services:

Service	Algorithms	Description
1. Authentication (digital signature)	DSS/SHA or RSA/SHA	Hash of message is created with SHA-1 and encrypted with sender's private key.
2. Confidentiality	CAST, IDEA, 3DES + RSA/ElGamal	Message encrypted with a one-time session key; session key encrypted with receiver's public key.
3. Compression	ZIP	Message compressed after signing and before encryption to save space.
4. Email compatibility	Radix-64	Binary data converted to ASCII characters for email transmission.
5. Segmentation	—	Long messages broken into segments to fit maximum email size.

Order: Sign → Compress → Encrypt → Radix-64 → Segment

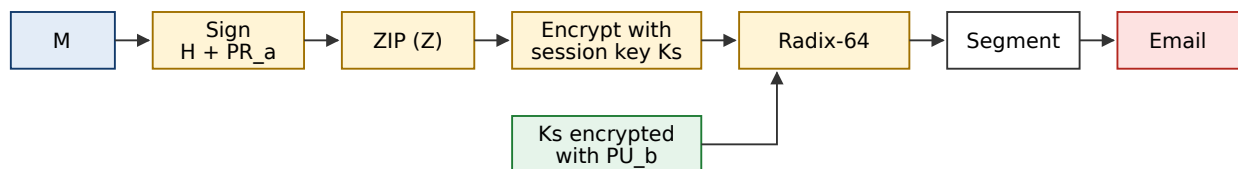


Fig 8.1 – PGP message generation (authentication + confidentiality)

Key management: PGP uses a **web of trust** instead of a central CA – users sign each other's public keys. Each user stores a **private-key ring** (own key pairs) and a **public-key ring** (others' public keys).

S/MIME (Secure/Multipurpose Internet Mail Extension)

S/MIME is a security enhancement to the MIME email format standard, based on RSA Data Security technology. It is an **industry standard** for commercial and organizational use (built into Outlook, Apple Mail etc.).

S/MIME functions:

- **Enveloped data:** encrypted content of any type plus encrypted session keys for one or more recipients (confidentiality).
- **Signed data:** message digest encrypted with the signer's private key; content + signature are base64-encoded – only S/MIME-capable receivers can view.
- **Clear-signed data:** only the signature is base64-encoded; receivers without S/MIME can still read the message.
- **Signed and enveloped data:** signed-only and encrypted-only entities may be nested.

Algorithms: SHA-1/SHA-256 for digest; DSS/RSA for signature; RSA/Diffie-Hellman for session-key encryption; AES/3DES for message encryption; HMAC for MAC. **Key management:** uses **X.509 v3 certificates** issued by CAs (hierarchical trust).

PGP	S/MIME
Mainly for personal email	Industry/organizational standard
Web of trust (users certify keys)	Hierarchical CA-based X.509 certificates
Radix-64 encoding	Base64 / MIME encoding
Separate software / plugin	Built into most email clients

Q9. What is a Firewall? Types and limitations

Firewall

A firewall is a hardware or software system placed between a **trusted internal network** and an **untrusted external network** (Internet) that monitors and controls incoming and outgoing traffic based on a defined **security policy**. It is a single choke point through which all traffic must pass.

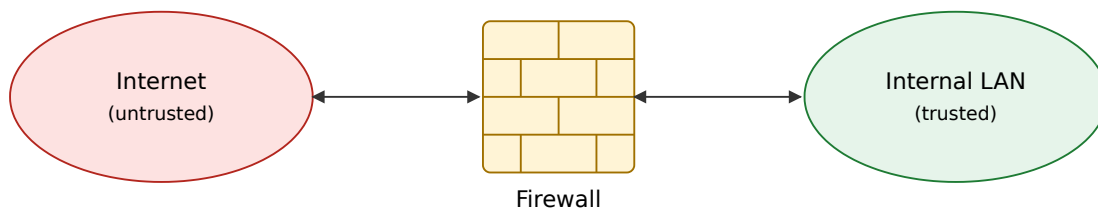


Fig 9.1 – Firewall between trusted and untrusted networks

Design goals: (1) all traffic from inside to outside and vice versa must pass through the firewall; (2) only authorized traffic, defined by local security policy, is allowed; (3) the firewall itself must be immune to penetration.

Firewall access techniques: service control, direction control, user control, behavior control.

Types of Firewalls

1. Packet Filtering Firewall (Network layer)

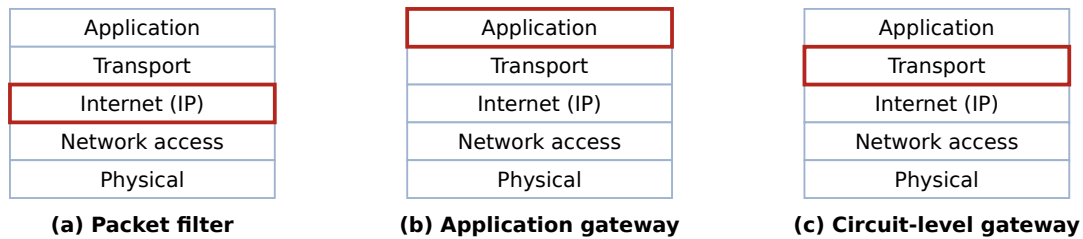
- Applies a set of rules to each incoming and outgoing IP packet and then forwards or discards it.
- Filtering is based on: source IP address, destination IP address, source and destination port numbers (TCP/UDP), IP protocol field, and router interface.
- Default policy: *discard* (what is not expressly permitted is prohibited) or *forward*.
- **Advantages:** simple, fast, transparent to users, cheap.
- **Disadvantages:** cannot inspect application data; vulnerable to IP spoofing, source routing and tiny-fragment attacks; limited logging.

Rule	Action	Src IP	Dst IP	Dst Port	Meaning
1	Allow	*	192.168.1.10	25	Inbound mail to mail server
2	Allow	192.168.1.*	*	80, 443	Internal users browse the web
3	Block	*	*	*	Default deny

2. Stateful Inspection Firewall – a packet filter that also keeps a **state table** of established TCP connections and allows incoming packets only if they belong to an existing connection.

3. Application-Level Gateway (Proxy server) – acts as a relay of application-level traffic (HTTP, FTP, SMTP). User contacts the gateway, which authenticates and then connects to the remote host on the user's behalf. More secure (can inspect content) but has high processing overhead.

4. Circuit-Level Gateway – sets up two TCP connections (inside ↔ gateway, gateway ↔ outside) and relays segments without examining contents. Example: SOCKS.



Red box = layer at which the firewall operates

Fig 9.2 – Layers at which firewall types operate

Firewall configurations: screened host (single-homed / dual-homed bastion), screened subnet (DMZ) firewall. A **bastion host** is a hardened system that serves as a platform for application/circuit gateways.

Limitations of Firewalls

1. Cannot protect against attacks that **bypass the firewall** (e.g., dial-up/modem connections, mobile hotspots).
2. Cannot protect against **internal threats** such as a disgruntled employee.
3. Cannot fully protect against transfer of **virus-infected files** or programs.
4. An improperly secured wireless LAN or a laptop infected outside can be used inside the network.
5. Cannot protect against **social engineering** and phishing attacks.
6. Misconfigured rules may give a false sense of security; can become a performance bottleneck.

Q10. Malicious software; types of Viruses, life cycle and countermeasures

Malicious Software (Malware)

Malware is a program inserted into a system, usually covertly, with the intent of compromising the **confidentiality, integrity or availability** of the victim's data, applications or operating system.

Classification: (a) those that need a host program (viruses, logic bombs, backdoors) and (b) independent programs (worms, bots); also non-replicating vs replicating.

Malware	Description
Virus	Attaches itself to a program and replicates when the host is executed.
Worm	Self-contained program that spreads across networks on its own.
Trojan horse	Program that appears useful but has hidden malicious function.
Logic bomb	Code that triggers when a condition is met (e.g., a specific date).
Backdoor / Trapdoor	Secret entry point bypassing normal authentication.
Rootkit	Tools to gain and hide administrator-level access.
Spyware / Keylogger	Collects information (keystrokes, browsing) secretly.
Ransomware	Encrypts user data and demands ransom.
Bot / Zombie	Infected machine controlled remotely; part of a botnet for DDoS/spam.

Virus

A virus is software that can **infect other programs by modifying them** to include a copy of itself. It has three parts: **infection mechanism** (how it spreads), **trigger** (event that activates payload) and **payload** (what it does).

Life cycle (phases) of a Virus

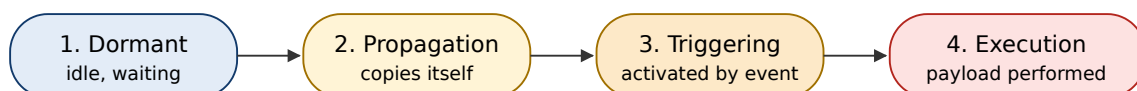


Fig 10.1 - Phases of a virus

1. **Dormant phase:** virus is idle; will be activated by some event (a date, presence of a file). Not all viruses have this stage.
2. **Propagation phase:** virus places a copy of itself into other programs or system areas on disk; each infected program now contains a clone.
3. **Triggering phase:** virus is activated to perform its intended function, caused by system events (e.g., count of copies, a date).
4. **Execution phase:** the payload is performed – harmless (message on screen) or damaging (destroying files).

Types of Viruses

By target:

- **Boot sector infector:** infects the master boot record and spreads when the system boots from the infected disk.
- **File infector:** infects executable files (.exe, .com).
- **Macro virus:** infects documents with macro code (MS Word/Excel); platform-independent and spreads via email.
- **Multipartite virus:** infects files in multiple ways (boot sector + files).

By concealment strategy:

- **Encrypted virus:** body encrypted with a random key; small decryption routine stored in clear.
- **Stealth virus:** designed to hide from antivirus software (e.g., compression, intercepting I/O).
- **Polymorphic virus:** mutates with every infection, making signature detection impossible.
- **Metamorphic virus:** rewrites itself completely at each iteration, changing behavior and appearance.
- **Memory-resident virus:** lodges in main memory and infects every program that executes.

Countermeasures

Ideal approach is **prevention**; when that fails, use:

1. **Detection:** determine that infection has occurred and locate the virus.
2. **Identification:** identify the specific virus.
3. **Removal:** remove all traces of the virus and restore the program to its original state.

Generations of anti-virus software:

- **1st generation - Simple scanners:** use virus signatures; detect only known viruses.
- **2nd generation - Heuristic scanners:** look for probable infection using rules, integrity checking (checksums).
- **3rd generation - Activity traps:** memory-resident programs that identify a virus by its actions.
- **4th generation - Full-featured protection:** combination of scanning, activity traps and access control.

Advanced techniques: Generic decryption (GD) using CPU emulator, Digital Immune System (DIS), Behavior-blocking software, sandboxing.

General good practices: keep OS and software patched; use updated antivirus; don't open unknown attachments; regular backups; firewall; least privilege; user awareness training.

Q11. Intrusion Detection System (IDS); Intrusion techniques; Honeypots

Intruders

An intruder is a person who gains or tries to gain unauthorized access to a system. Three classes (Anderson):

- **Masquerader:** an outsider who uses a legitimate user's account.
- **Misfeasor:** a legitimate user who accesses data/resources they are not authorized for, or misuses privileges.
- **Clandestine user:** one who seizes supervisory control to evade auditing and access controls.

Intrusion Techniques

The objective is to gain access or increase privileges. Most techniques target **passwords**:

1. Try default passwords shipped with systems (e.g., admin/admin).
2. Exhaustively try all short passwords (brute force).
3. Try words from a dictionary or list of likely passwords (dictionary attack).
4. Collect user information – names, family, birthdays, hobbies – and try them.
5. Try users' phone numbers, roll numbers, vehicle numbers.
6. Use a **Trojan horse** to capture passwords; keyloggers.
7. **Tap the line** between a remote user and host (sniffing).
8. Social engineering and phishing; exploiting software vulnerabilities (buffer overflow).

Intrusion Detection System (IDS)

An IDS is a hardware/software system that **monitors system or network activity** for malicious actions or policy violations and raises alerts. It is the second line of defense after firewalls.

Why IDS: quickly detect and eject intruders before damage; acts as a deterrent; collects information about intrusion techniques to strengthen prevention.

Components: Sensors (collect data), Analyzers (determine if intrusion occurred), User interface (view output / control).

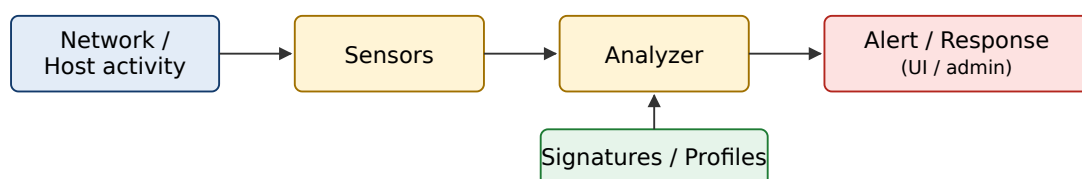


Fig 11.1 – Generic IDS architecture

Types based on source:

- **Host-based IDS (HIDS):** monitors a single host – audit logs, file integrity, system calls.
- **Network-based IDS (NIDS):** monitors network traffic at selected points (sensors) – packet headers and payloads.

Types based on detection approach:

- **Statistical anomaly detection:** builds a profile of normal behavior (threshold detection, profile-based); flags deviations. Can detect new attacks but has more false positives.
- **Rule-based / signature (misuse) detection:** uses known attack patterns (signatures). Accurate for known attacks; cannot detect new ones.

Audit records (native or detection-specific) are the fundamental tool: subject, action, object, exception condition, resource usage, timestamp.

False positive – legitimate user flagged as intruder; **False negative** – intruder not detected.

Honeypots

A honeypot is a **decoy system** designed to lure potential attackers away from critical systems. It contains fabricated information that looks valuable, but no legitimate user would access it – so **any access is suspicious**.

Goals:

- Divert an attacker from accessing critical systems.
- Collect information about the attacker's activity and techniques.
- Encourage the attacker to stay long enough for administrators to respond.

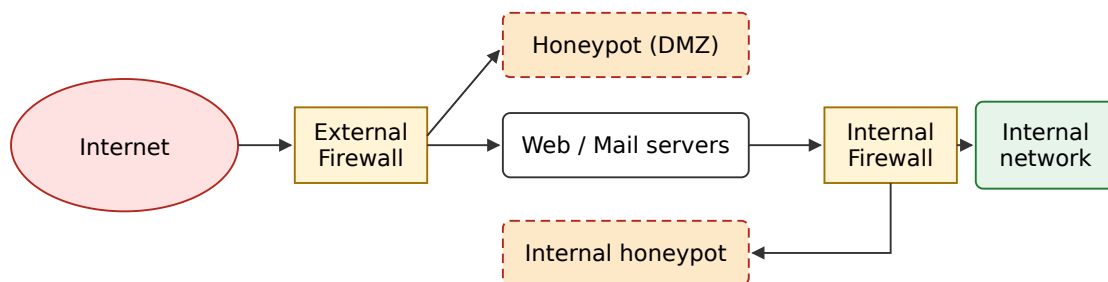


Fig 11.2 – Example honeypot deployment locations

Types: **Low-interaction honeypot** (emulates services, easy to deploy, less information) and **High-interaction honeypot** (real system with full OS, more realistic, more risk). A network of honeypots is called a **honeynet**.

Limitations: only detects attacks directed at it; attacker may use a compromised honeypot to attack other systems; may be identified by skilled attackers.

Q12. IPSec architecture and modes of operation

IP Security (IPSec)

IPSec is a framework of open standards (IETF) that provides security at the **IP (network) layer** – authentication, integrity, confidentiality and key management for both IPv4 and IPv6. Because it works at the network layer it is transparent to applications and end users. Used for VPNs, secure branch office connectivity and remote access.

Services: access control, connectionless integrity, data origin authentication, rejection of replayed packets, confidentiality (encryption), limited traffic-flow confidentiality.

IPSec Architecture (Document overview)

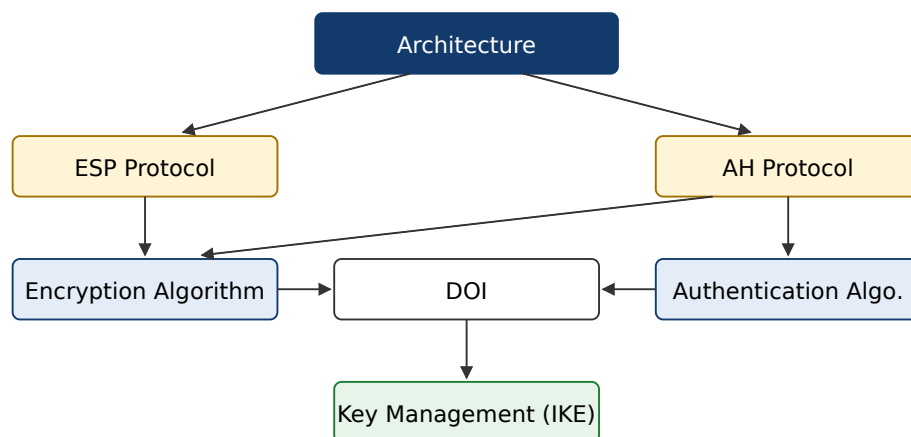


Fig 12.1 – IPSec document roadmap / architecture

1. **Architecture:** general concepts, security requirements, definitions and mechanisms.
2. **Authentication Header (AH):** provides data integrity, data-origin authentication and anti-replay – **no encryption**. Fields: Next header, Payload length, Reserved, SPI, Sequence number, Integrity Check Value (ICV).
3. **Encapsulating Security Payload (ESP):** provides **confidentiality (encryption)** plus optional authentication. Fields: SPI, Sequence number, Payload data, Padding, Pad length, Next header, ICV.
4. **Encryption algorithms:** used by ESP (AES, 3DES, CBC mode).
5. **Authentication algorithms:** HMAC-SHA-1/256, HMAC-MD5.
6. **Key management: IKE** (Internet Key Exchange) – Oakley + ISAKMP – for automatic key exchange and negotiation of SAs.
7. **Domain of Interpretation (DOI):** values needed to relate the documents (identifiers for algorithms, key lifetimes).

Security Association (SA): a one-way logical relationship between sender and receiver that affords security services. Identified by: **Security Parameter Index (SPI)**, **IP destination address** and **security protocol identifier** (AH or ESP). SAs are stored in the **SAD** (Security Association Database); policies in the **SPD** (Security Policy Database).

Modes of Operation

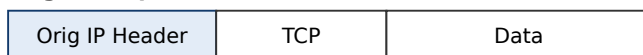
1. Transport Mode

- Protects the **payload of the IP packet** (upper-layer protocol data such as TCP/UDP segment); the original IP header is not protected/encrypted.
- Used for **end-to-end** communication between two hosts (client-server).
- ESP in transport mode encrypts (and optionally authenticates) the IP payload; AH authenticates the payload and selected portions of the IP header.

2. Tunnel Mode

- Protects the **entire original IP packet**; the whole packet (with original header) becomes the payload of a **new outer IP packet** with a new header.
- Used between **security gateways** (firewalls/routers) – e.g., **VPNs**; hosts behind gateways need not implement IPSec.
- Hides the original source and destination addresses → traffic-flow confidentiality.

Original IP packet

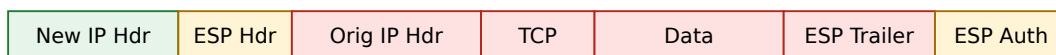


= encrypted

Transport mode (ESP)



Tunnel mode (ESP)



Authenticated part = ESP header through ESP trailer

Fig 12.2 – ESP in transport mode vs tunnel mode

Transport Mode	Tunnel Mode
Protects only IP payload	Protects entire IP packet
Original IP header used	New outer IP header added
Host-to-host (end-to-end)	Gateway-to-gateway / host-to-gateway (VPN)
Less overhead	More overhead
Source/destination visible	Original addresses hidden

Q13. Secure Electronic Transaction (SET) protocol; SSL working mechanism

Secure Electronic Transaction (SET)

SET is an open encryption and security specification developed by **Visa and MasterCard** (1996) to protect **credit card transactions** on the Internet. It is not a payment system itself, but a set of security protocols and formats.

Services / Requirements: confidentiality of payment and order information; integrity of transmitted data; authentication of cardholder as legitimate user; authentication of merchant; uses best security practices; independent of transport security.

SET participants:

1. **Cardholder** – authorized holder of a payment card.
2. **Merchant** – person/organization selling goods or services.
3. **Issuer** – financial institution (bank) that issued the card.
4. **Acquirer** – financial institution that establishes the merchant account and processes card authorizations and payments.
5. **Payment Gateway** – operated by the acquirer; processes merchant payment messages.
6. **Certification Authority (CA)** – issues X.509 v3 certificates to cardholders, merchants and gateways.

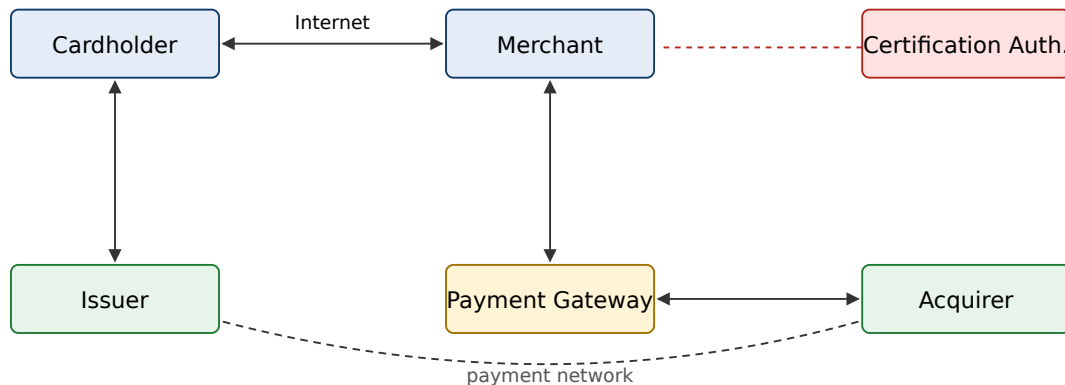


Fig 13.1 – SET participants

Transaction sequence: (1) Customer opens account; (2) Customer receives certificate; (3) Merchants have their own certificates; (4) Customer places an order; (5) Merchant is verified; (6) Order and payment are sent; (7) Merchant requests payment authorization; (8) Merchant confirms order; (9) Merchant provides goods/services; (10) Merchant requests payment.

Dual Signature (key innovation of SET): links the Order Information (OI) for the merchant and the Payment Information (PI) for the bank, such that **the merchant cannot see the card details and the bank cannot see the order details**, yet the link between the two can be proved.

$$DS = E(PR_c, H(H(PI) \parallel H(OI)))$$
$$PIMD = H(PI), OIMD = H(OI), POMD = H(PIMD \parallel OIMD)$$

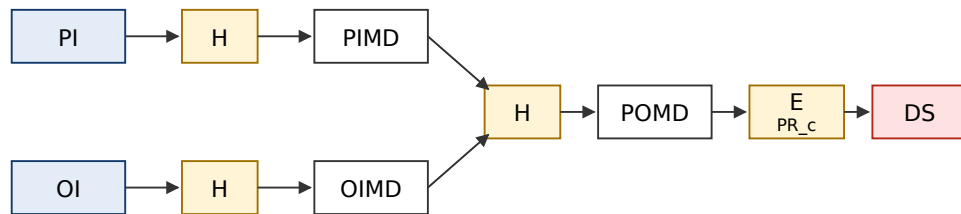


Fig 13.2 - Construction of the dual signature

SSL (Secure Sockets Layer) - Working Mechanism

SSL (Netscape) provides security services between TCP and applications (HTTP, FTP, SMTP). Its successor is **TLS**. HTTPS = HTTP over SSL/TLS. SSL is a two-layer set of protocols:

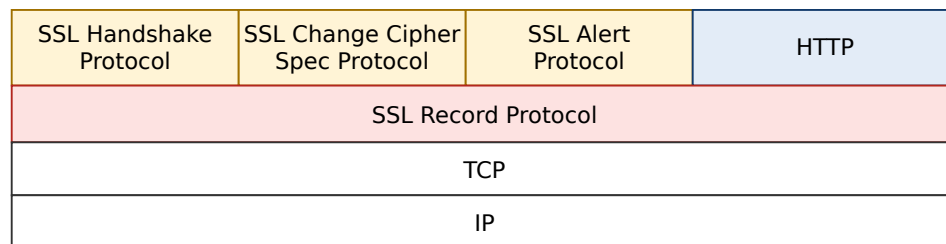


Fig 13.3 - SSL protocol stack

Two important concepts: Connection (a transient peer-to-peer transport) and **Session** (an association between client and server created by Handshake, defining a set of cryptographic parameters shared among multiple connections).

1. SSL Record Protocol - provides **confidentiality** (symmetric encryption) and **message integrity** (MAC). Steps: **Fragmentation** ($\leq 2^{14}$ bytes) → **Compression** (optional) → **Add MAC** → **Encrypt** → **Append SSL record header**.

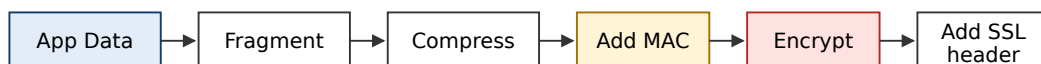


Fig 13.4 - SSL Record Protocol operation

2. Change Cipher Spec Protocol - a single byte message with value 1; causes the pending state to be copied into the current state (activates the newly negotiated cipher suite).

3. Alert Protocol - conveys SSL-related alerts; 2 bytes: level (warning = 1 / fatal = 2) and alert code (e.g., unexpected_message, bad_record_mac, handshake_failure).

4. Handshake Protocol - the most complex part; lets server and client **authenticate each other** and **negotiate encryption/MAC algorithms and keys**. Four phases:

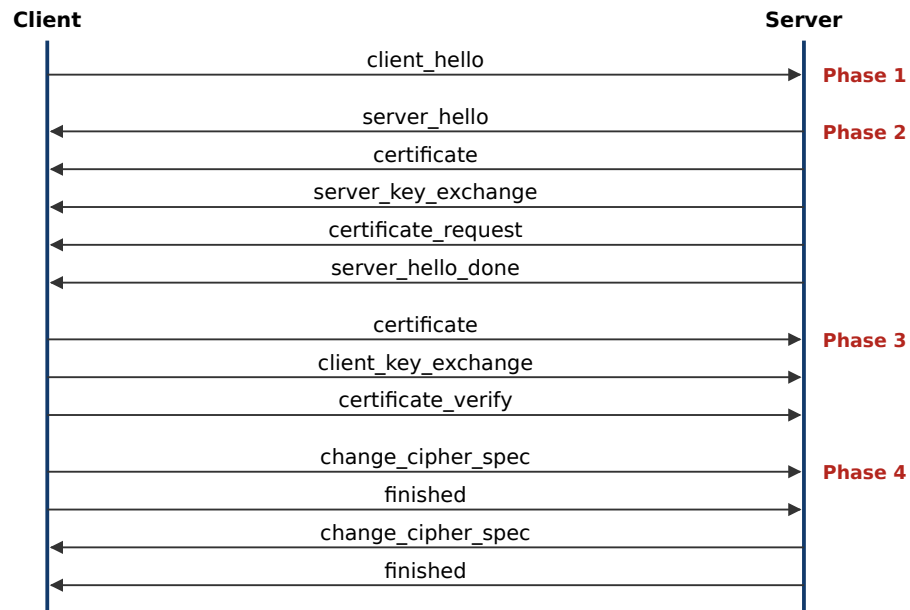


Fig 13.5 - SSL Handshake Protocol

- Phase 1 - Establish security capabilities:** client_hello (version, random, session ID, cipher suite list, compression) and server_hello (chosen parameters).
- Phase 2 - Server authentication and key exchange:** server sends its certificate, key exchange, optional certificate request, then server_hello_done.
- Phase 3 - Client authentication and key exchange:** client sends certificate (if requested), client_key_exchange (pre-master secret encrypted with server's public key), certificate_verify.
- Phase 4 - Finish:** change_cipher_spec and finished messages exchanged by both sides; secure connection established.

SET	SSL
Specifically for credit-card payment	General-purpose secure channel
Uses dual signature; merchant can't see card number	Merchant receives card details
All parties need certificates	Usually only server certificate
Complex, slow, not widely adopted	Simple, widely used (HTTPS)

Q14. Define Cryptanalysis, Brute Force Attack and the Avalanche effect

Cryptanalysis

Cryptanalysis is the science of **breaking ciphers** – recovering the plaintext or the key from ciphertext **without knowing the secret key**, by exploiting the characteristics of the algorithm, some knowledge of plaintext, or plaintext-ciphertext pairs. Cryptography + Cryptanalysis = **Cryptology**.

Types of cryptanalytic attacks (based on information available to the attacker):

Attack	Known to cryptanalyst
Ciphertext only	Encryption algorithm, ciphertext
Known plaintext	Algorithm, ciphertext, one or more plaintext-ciphertext pairs
Chosen plaintext	Algorithm, ciphertext, plaintext chosen by the attacker with its ciphertext
Chosen ciphertext	Algorithm, ciphertext, ciphertext chosen by attacker with decrypted plaintext
Chosen text	Combination of chosen plaintext and chosen ciphertext

Other techniques: frequency analysis, differential cryptanalysis, linear cryptanalysis, side-channel (timing) attacks.

An encryption scheme is **computationally secure** if the cost of breaking it exceeds the value of the information, or the time required exceeds the useful lifetime of the information.

Brute Force Attack

A brute force attack is one in which the attacker **tries every possible key** on a piece of ciphertext until an intelligible plaintext is obtained. On average, **half of all possible keys** must be tried to achieve success. The defense is a **large key size**.

Key size (bits)	Number of keys	Time at 10^9 decryptions/sec	Example
56	$2^{56} \approx 7.2 \times 10^{16}$	≈ 1.125 years	DES
128	$2^{128} \approx 3.4 \times 10^{38}$	$\approx 5.3 \times 10^{21}$ years	AES-128
168	$2^{168} \approx 3.7 \times 10^{50}$	$\approx 5.8 \times 10^{33}$ years	3DES
256	$2^{256} \approx 1.2 \times 10^{77}$	$\approx 1.8 \times 10^{60}$ years	AES-256

Example: Caesar cipher has only 25 keys – all can be tried in seconds. In password systems, brute force tries all combinations; countermeasures include long complex passwords, account lockout, CAPTCHA, and salting with slow hashes.

Avalanche Effect

The avalanche effect is a desirable property of encryption algorithms and hash functions in which **a small change in the plaintext or the key (even a single bit) produces a significant change in the ciphertext** – ideally about **half of the output bits** change.



Fig 14.1 - Avalanche effect (illustrative values)

- In **DES**, changing one bit of the plaintext changes about 34 of the 64 ciphertext bits after 16 rounds; most of the change appears within the first few rounds.
- In hash functions: e.g., SHA-256("hello") and SHA-256("Hello") produce completely different digests.
- **Importance:** if a cipher lacks the avalanche effect, a cryptanalyst can predict the input from the output pattern, making the cipher weak. It results from good **confusion** (S-boxes) and **diffusion** (permutations/MixColumns).
- **Strict Avalanche Criterion (SAC):** each output bit should change with probability $\frac{1}{2}$ whenever any single input bit is changed.

Exam tip: For 5-mark answers, write the definition, a labelled diagram and 4–5 key points. For numericals (Q2, Q3), always show each step – key generation, intermediate values and the final verification.